

Análisis y Diseño con Patrones

PRA 2: Patrones de Diseño y de Asignación de Responsabilidades

En esta segunda práctica trabajaremos sobre un caso basado en una plataforma para la gestión de eventos sociales, la cual servirá de marco de trabajo para la aplicación de patrones de diseño y de asignación de responsabilidades.

 **NOTA:** Todos los modelos UML que se entreguen deben estar realizados mediante una herramienta de modelado. No se aceptan modelos realizados a mano, ni con herramientas de dibujo tipo *Paint*, ya que estos modelos no tienen rigor. Se recomienda usar la herramienta [VisualParadigm community edition](https://visualparadigm.com/community-edition/).

 **FORMATO:** El documento que entregues con tu propuesta de solución para esta PRA **debe seguir el siguiente formato** (el texto en cursiva corresponde a información que debes rellenar):

Alumno: *...tu nombre...*

Compromiso:

...Certifico que...

Pregunta 1.1:

...tu propuesta de solución...

 **NOTA:** Las preguntas de esta PRA no permiten una exactitud absoluta respecto a la realidad técnica de la plataforma, dada la falta de información detallada. El enfoque debe centrarse en proponer soluciones plausibles que sean coherentes con los materiales de la asignatura y el funcionamiento general del sistema. El criterio principal de evaluación será la justificación lógica de las decisiones tomadas, otorgando mayor valor al razonamiento argumentado.

 **RECUERDA:** Tal como se explica en el Plan docente de la asignatura, esta actividad debe resolverse de forma individual. Por otro lado, no está permitido el uso de herramientas de inteligencia artificial. **En caso de detectar copias o uso de herramientas de IA, se penalizará la actividad con 0 % como nota.**

Para garantizar que has resuelto esta actividad de forma individual y sin uso de herramientas de IA, te pedimos que en tu solución entregada añadas el siguiente compromiso de autorresponsabilidad:

Certifico que he realizado la PRA2 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento "Originalidad en la evaluación" disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de 0 %.

Compromiso de autorresponsabilidad

Escribe aquí el texto con el compromiso de autorresponsabilidad que consta en cursiva en la nota previa. **No escribirlo implica la no corrección de la PRA y una calificación de 0 %.**

Certifico que he realizado la PRA2 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento “Originalidad en la evaluación” disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de 0 %.

Caso práctico

Se desea desarrollar una aplicación para la gestión de eventos sociales (como speed dating, intercambios de idiomas o *networking*), donde los usuarios pueden registrarse, participar en eventos y establecer conexiones con otros asistentes. La plataforma debe permitir gestionar usuarios, eventos, participaciones, preferencias y resultados de interacción entre participantes.

Las clases principales del sistema serían:

- *User*: Un usuario puede registrarse en múltiples eventos y expresar interés (*like*) por otros participantes. Cuenta con un identificador, un nombre, edad, email, un booleano para indicar si es usuario premium o no y una lista de intereses.
- *Event*: Representa un evento organizado en la plataforma. Un evento tiene una capacidad limitada y un conjunto de participantes registrados. Puede cambiar de estado según su evolución. Cuenta con un identificador, un título, una localización, una fecha, una capacidad o aforo máximo, un estado (*OPEN*, *FULL*, *CLOSED*) y el tipo de evento (*SPEEDDATING*, *AFTERWORK*, *NETWORKING*, *LANGUAGE_EXCHANGE* o *PREMIUM*).
- *Registration*: Permite modelar la relación entre usuarios y eventos, así como información adicional de la participación. Ese registro tendrá un identificador, una fecha de registro y un booleano para indicar si el usuario ha hecho check-in en ese evento (si ha aparecido).
- *Like*: Representa la preferencia de un usuario hacia otro dentro de un evento. Un usuario puede indicar interés por otros participantes del mismo evento. Cada *like* tendrá un identificador.
- *Match*: Representa una coincidencia entre dos usuarios. Un *match* se genera cuando existe interés mutuo entre dos usuarios. Cada *match* tendrá un identificador, así como una fecha.

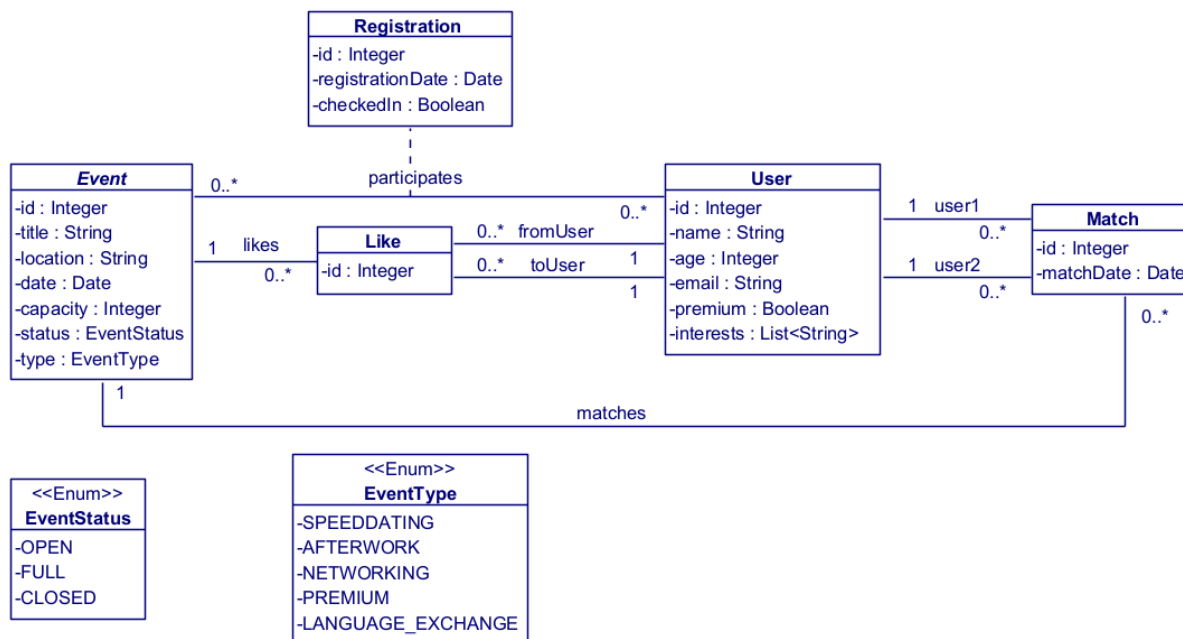
Un *User* puede participar en varios *Event*. Un *Event* contiene múltiples *User* (participantes). Un *User* puede dar múltiples *Like* dentro de un *Event* y finalmente un *Match* se genera a partir de dos *Like* recíprocos.

El sistema debe permitir:

- Gestionar eventos y participantes
- Registrar interacciones entre usuarios
- Generar resultados (*matches*)
- Notificar a los usuarios
- Ofrecer funcionalidades avanzadas (premium)

A partir de este contexto, se plantean distintas necesidades de evolución del sistema. El objetivo será **analizar cada situación y proponer soluciones de diseño adecuadas**, justificando el uso de patrones.

Aquí tenéis el diagrama UML de clases del sistema:



Restricciones de integridad:

- User.id, Event.id, Registration.id, Like.id y Match.id son únicos.
- Toda Registration debe estar asociada a exactamente 1 User y 1 Event.
- Todo Like debe estar asociado a 1 Event y a 1 fromUser y a 1 toUser donde fromUser y toUser no pueden ser el mismo usuario y deben ser del mismo Event.
- Todo Match debe estar asociado a exactamente 2 Users y a 1 Event. Los dos usuarios vinculados al match deben ser diferentes. Un match solo existe si hay reciprocidad y no puede haber duplicados: $\text{Match}(u1, u2) = \text{Match}(u2, u1)$.
- $\text{Event.participants.size} \leq \text{Event.capacity}$.
- Un User no puede registrarse 2 veces en el mismo Event.

Pregunta 1 (25 %)

A partir del modelo inicial de la aplicación, formado por las clases *User*, *Event*, *Registration*, *Like* y *Match*, se desea incorporar una nueva funcionalidad para generar automáticamente los resultados de un evento una vez este haya finalizado.

Actualmente, un *Match* se crea cuando dos usuarios se han dado “like” mutuamente dentro del mismo *Event*. Sin embargo, la organización prevé que no todos los eventos utilicen siempre el mismo criterio para calcular los matches.

Por ejemplo, en algunos eventos se podrían aplicar las siguientes reglas:

- En eventos de tipo **speed dating**, solo se generará un match si existe reciprocidad directa entre dos participantes.
- En eventos **premium**, además de la reciprocidad, se tendrá en cuenta la edad, intereses comunes y prioridad de usuarios premium.
- En eventos de **networking**, se podrían generar sugerencias aunque no exista reciprocidad completa, siempre que haya un grado suficiente de afinidad.
- En eventos futuros, podrían incorporarse nuevos criterios de cálculo sin modificar el código existente de las entidades principales.

Además, el responsable técnico indica que no quiere que la clase *Event* ni la clase *Match* contengan estructuras condicionales complejas del tipo:

```
if (eventType.equals("SPEED_DATING")) {
    ...
} else if (eventType.equals("PREMIUM")) {
    ...
} else if (eventType.equals("NETWORKING")) {
    ...
}
```

Pregunta 1.1 (2.5 %). Para llevar a cabo la implementación de este caso de uso de forma eficiente y óptima, ¿qué patrón o patrones de diseño emplearías? Justifica la respuesta brevemente.

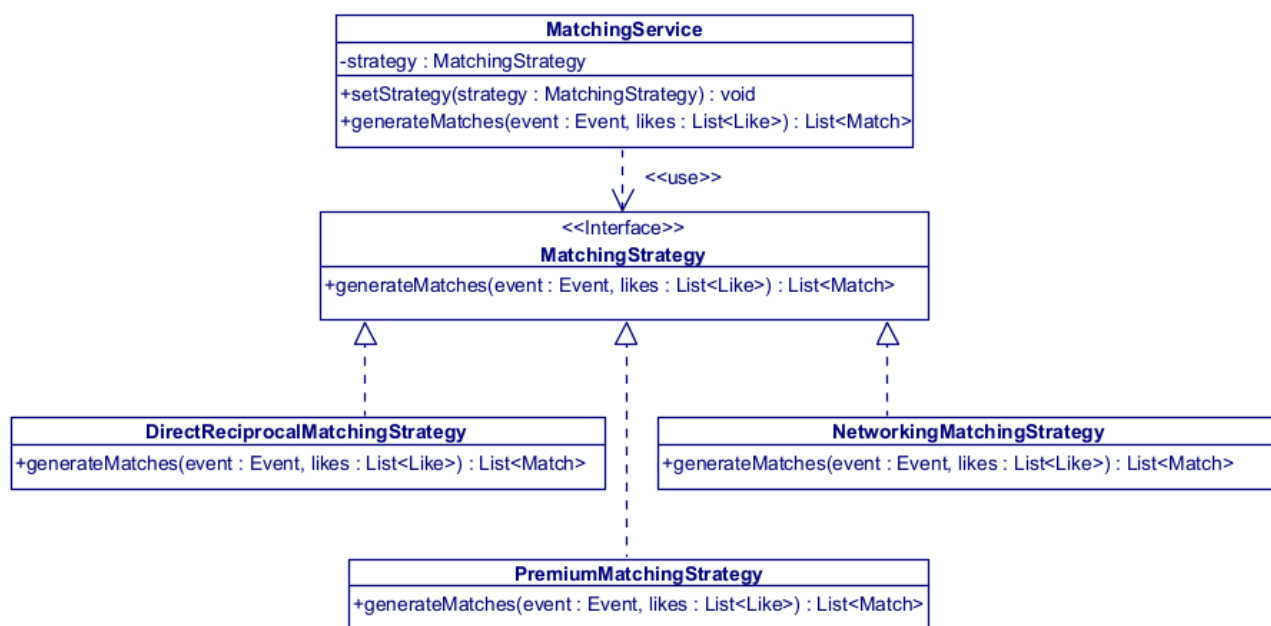
Pregunta 1.2 (15 %). Crea el diagrama de clases UML donde se vea claramente la aplicación del patrón o patrones empleados.

Pregunta 1.3 (7,5 %). Escribe pseudocódigo que muestre cómo el sistema invoca el proceso de generación de matches de un evento desde el servicio asociado utilizando tu solución propuesta. No es necesario presentar el pseudocódigo de cómo se calculan al detalle exacto los matches en base a los likes del evento, sino que lo que interesa es que, mediante el pseudocódigo, se vea claramente la utilización del patrón o patrones usados para determinar qué regla utilizar. ¿Sería posible cambiar la forma de calcular los matches en tiempo de ejecución? ¿Cómo?

Solución

Pregunta 1.1. El patrón adecuado es **Strategy**. Se aplica porque el sistema necesita **variar el algoritmo de generación de matches** según el tipo de evento, evitando llenar *Event* o *Match* de condicionales. Este patrón permite encapsular cada algoritmo de matching en una clase diferente y hacerlos intercambiables.

Pregunta 1.2. El diagrama de clases sería:



Pregunta 1.3. La estrategia podría ser cambiada dinámicamente en tiempo de ejecución en el método “setStrategy”.

```

public interface MatchingStrategy {
    public generateMatches(Event event, List<Like> likes) : List<Match>;
}

public class MatchingService{
    private strategy: MatchingStrategy;
    public MatchingService(strategy : MatchingStrategy) {
        this.strategy = strategy;
    }
    // Allow changing the strategy at runtime
    public setStrategy(newStrategy : MatchingStrategy) : void {
        this.strategy = newStrategy;
    }
    public generateMatches(Event event, List<Like> likes) : List<Match> {

```

```
        return strategy.generateMatches(event, likes);
    }
}

public class DirectReciprocalMatchingStrategy implements MatchingStrategy {
    public generateMatches(Event event, List<Like> likes) : List<Match> {
        // Calculation of event matches based on likes.
    }
}
```

Ejemplo de cómo se llamaría:

```
MatchingStrategy strategy = new DirectReciprocalMatchingStrategy();
MatchingService service = new MatchingService(strategy);
List<Match> matches = service.generateMatches(event, likes);
```

Si luego interesa en tiempo de ejecución cambiar la estrategia, se puede hacer:

```
MatchingStrategy strategy2 = new PremiumMatchingStrategy();
service.setStrategy(strategy2);
List<Match> matches2 = service.generateMatches(event, likes);
```

Pregunta 2 (30 %)

Los arquitectos técnicos prevén que la plataforma evolucione y que se incorporen nuevos tipos de actividades. Además, consideran imprescindible que los diferentes tipos de eventos puedan tener reglas y datos específicos.

La organización quiere incorporar tres modalidades principales:

- *SpeedDatingEvent*: Representa un evento orientado a citas rápidas. Puede incluir información específica como el rango de edad, la duración del evento y el número máximo de matches que un usuario puede tener.
- *LanguageExchangeEvent*: Representa un evento de intercambio de idiomas. Puede incluir información específica como: idioma principal, idioma secundario y nivel.
- *NetworkingEvent*: Representa un evento profesional o de contactos. Puede incluir información específica como el sector, tema profesional y un booleano que indique si se requiere ser autónomo.

Actualmente, los eventos se crean desde diferentes partes del sistema: panel de administración, importación desde un formulario externo y herramienta interna de planificación. En cada una de estas partes se está empezando a repetir la lógica. El equipo técnico quiere evitar que esta lógica de creación quede repartida por toda la aplicación. Además, se prevé que en el futuro puedan aparecer nuevos tipos de evento, como: *BoardGameEvent*, *ConcertEvent* e *InvestmentTalkEvent* y no se desea modificar todas las pantallas o servicios que actualmente crean eventos.

Pregunta 2.1 (5 %). Analiza el problema de diseño que aparece al tener distintos tipos de evento y múltiples puntos del sistema donde se crean objetos. Indica qué patrón de diseño aplicarías para encapsular y centralizar la creación de los diferentes tipos de evento. Justifica por qué este patrón es más adecuado que, por ejemplo, añadir condiciones if/else directamente.

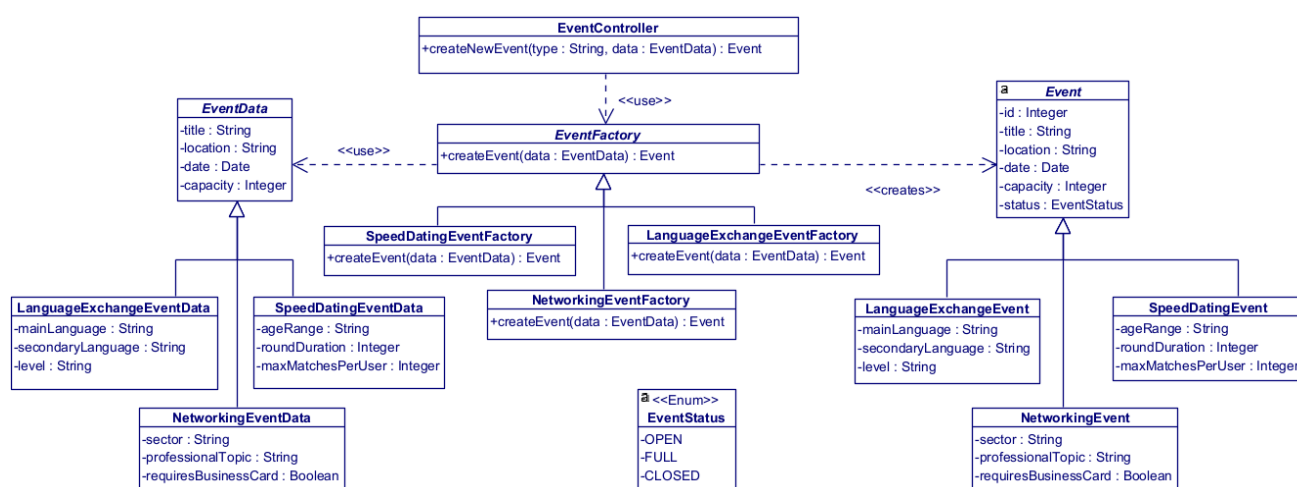
Pregunta 2.2 (15 %). Propón un diagrama de clases UML teniendo en cuenta que por el momento tendremos tres tipos de eventos. No hay que dibujar todas las clases del sistema; solo las necesarias para entender el patrón o patrones de diseño aplicados.

Pregunta 2.3 (10 %). Escribe el pseudocódigo simplificado de la parte del sistema mostrando cómo se invoca la creación del tipo de evento. No hay que implementar toda la lógica del caso de uso, solo la parte relacionada con la creación del evento según su tipo.

Solución

Pregunta 2.1. El patrón adecuado es **Factory Method**, ya que permite centralizar la creación de objetos *Event* sin que los clientes tengan que conocer las clases concretas. De esta forma se cumple con el principio Open-Closed, facilita la escalabilidad y mantenimiento de la aplicación y evita repetir lógica del código en diferentes partes de la aplicación, cosa que no sería posible de cumplir con el uso de condiciones if/else directamente.

Pregunta 2.2. El diagrama de clases sería:



Pregunta 2.3.

Se omiten algunos constructores y métodos get/set por simplificación del pseudocódigo.

```

public abstract class EventData {
    private title: String;
    private location: String;
    private date: Date;
    private capacity: Integer;

    protected createEvent(data: EventData) {
        this.title = data.getTitle();
        this.location = data.getLocation();
        this.date = data.getDate();
        this.capacity = data.getCapacity();
    }
}

public class SpeedDatingEventData extends EventData {
    private ageRange: String;

```

```
        private roundDuration: Integer;
        private maxMatchesPerUser: Integer;
    }

    public class LanguageExchangeEventData extends EventData {
        private mainLanguage: String;
        private secondaryLanguage: String;
        private level: String;
    }

    public class NetworkingEventData extends EventData {
        private sector: String;
        private professionalTopic: String;
        private requiresBusinessCard: Boolean;
    }

    public abstract class EventFactory {
        public abstract createEvent(data: EventData) : Event;
    }

    public class SpeedDatingEventFactory extends EventFactory {
        public createEvent(data: EventData) : Event {

            if (!(data instanceof SpeedDatingEventData)) {
                throw new Exception("Invalid data for SpeedDatingEvent");
            }
            SpeedDatingEventData sdData = (SpeedDatingEventData) data;
            SpeedDatingEvent event = new SpeedDatingEvent();
            event.setAgeRange(sdData.getAgeRange());
            event.setRoundDuration(sdData.getRoundDuration());
            event.setMaxMatchesPerUser(sdData.getMaxMatchesPerUser());
            super.createEvent(data);
            return event;
        }
    }

    public class LanguageExchangeEventFactory extends EventFactory {
        public createEvent(data: EventData) : Event {
            if (!(data instanceof LanguageExchangeEventData)) {
                throw new Exception("Invalid data for LanguageExchangeEvent");
            }
            LanguageExchangeEventData sdData = (LanguageExchangeEventData) data;
            LanguageExchangeEvent event = new LanguageExchangeEvent();
            event.setMainLanguage(sdData.getMainLanguage());
            event.setLevel(sdData.getLevel());
            event.setSecondaryLanguage(sdData.getSecondaryLanguage());
            super.createEvent(data);
            return event;
        }
    }
```

```
    }  
}  
  
public class NetworkingEventFactory extends EventFactory {  
    public createEvent(data: EventData) : Event {  
        if (!(data instanceof NetworkingEventData)) {  
            throw new Exception("Invalid data for NetworkingEvent");  
        }  
        NetworkingEventData sdData = (NetworkingExchangeEventData) data;  
        NetworkingEvent event = new NetworkingEvent();  
        event.setSector(sdData.getSector());  
        event.setProfessionalTopic(sdData.getProfessionalTopic());  
        event.setRequiresBusinessCard(sdData.getRequiresBusinessCard());  
        super.createEvent(data);  
        return event;  
    }  
}  
  
public class EventController {  
    public createNewEvent(type: String, data: EventData) : Event {  
        public factory: EventFactory;  
        if (type == "SPEED_DATING") {  
            factory = new SpeedDatingEventFactory();  
        } else if (type == "LANGUAGE_EXCHANGE") {  
            factory = new LanguageExchangeEventFactory();  
        } else if (type == "NETWORKING") {  
            factory = new NetworkingEventFactory();  
        } else {  
            throw new Exception("Unknown event type");  
        }  
        event: Event = factory.createEvent(data);  
        return event;  
    }  
}
```

Pregunta 3 (30 %)

Cada evento contiene un conjunto de participantes inscritos mediante objetos *Registration*. Hasta ahora, la aplicación únicamente necesitaba obtener la lista completa de asistentes para mostrarla en pantalla o generar los resultados del evento. Sin embargo, la organización quiere incorporar nuevas funcionalidades para mejorar la gestión interna de los eventos y ofrecer servicios adicionales a ciertos usuarios.

El equipo de producto ha detectado que los organizadores necesitan consultar los participantes de un evento de distintas formas sin exponer la estructura interna, según la situación. Hacen uso de la clase de gestión de eventos *EventManager* para obtener estos listados. Por ejemplo:

- Para preparar las mesas del speed dating, se necesita obtener los participantes en **orden de inscripción**.
- Para asignar turnos de conversación, se quiere obtenerlos en un **orden aleatorio**.
- Para revisar servicios especiales, se necesita filtrar únicamente por los usuarios que han contratado algún servicio adicional (es decir, son premium)

Paralelamente, la plataforma quiere ofrecer servicios opcionales a los participantes de un evento. Estos servicios pueden contratarse de forma independiente y combinarse entre sí. Algunos ejemplos son:

- Perfil destacado en la lista de participantes.
- Prioridad en la asignación de turnos.
- Mayor visibilidad en sugerencias de match.
- Acceso a estadísticas personales después del evento.

Un mismo usuario podría tener varios servicios al mismo tiempo. Se quiere que estos servicios puedan añadirse o retirarse sin modificar la clase *User* original, ya que esta clase forma parte del modelo base y se usa en muchas partes de la aplicación.

Pregunta 3.1 (5 %). Justifica qué patrón o patrones de diseño se deben aplicar para implementar la casuística expuesta en el enunciado de manera que el acoplamiento se mantenga bajo y se cumpla el principio de Open-Closed.

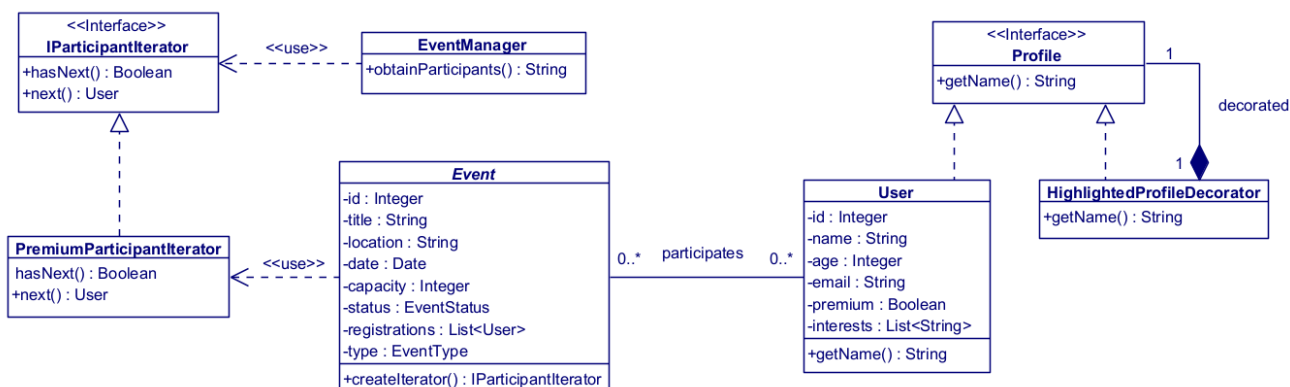
Pregunta 3.2 (15 %). Muestra el diagrama de clases con las operaciones necesarias para implementar este diseño.

Pregunta 3.3 (10 %). Muestra el diagrama de secuencia del siguiente caso simplificado: Se quieren obtener los participantes “premium” y pintar todos los asistentes al evento en un informe. A medida que se van obteniendo, como por defecto estos cuentan con el servicio de perfil destacado, el sistema debe añadir dinámicamente delante del nombre de cada usuario el prefijo “VIP USER” que corresponde con el servicio opcional de perfil destacado.

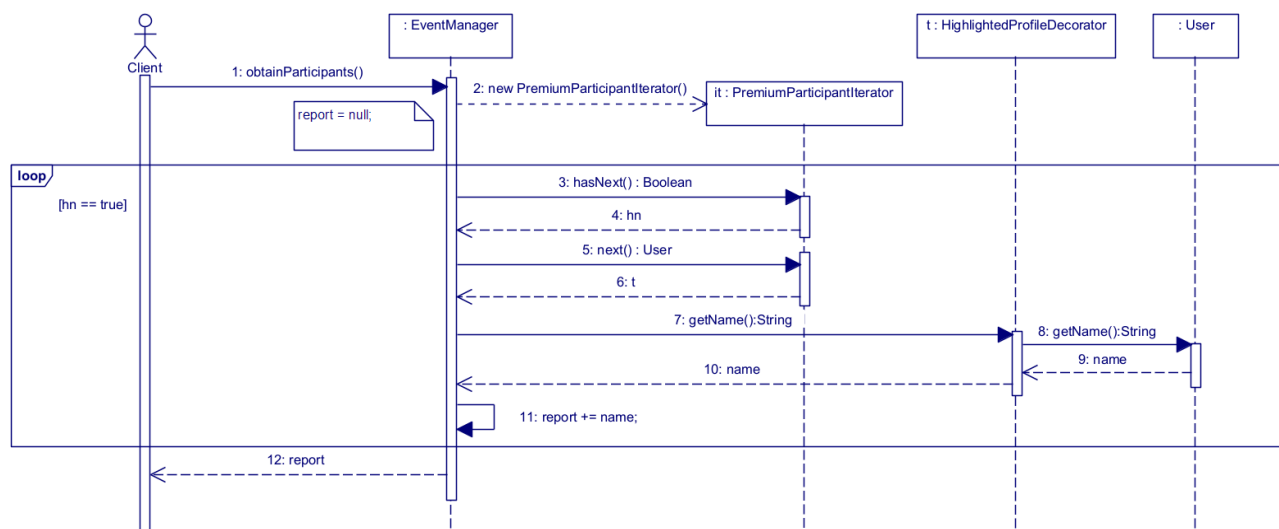
Solución

Pregunta 3.1. Los patrones adecuados son: **Iterator** para recorrer participantes sin exponer la estructura interna de *Event* y el patrón **Decorator** para añadir servicios opcionales a un *User* sin modificar la clase original ni crear muchas subclases.

Pregunta 3.2.



Pregunta 3.3. El diagrama de secuencia sería:



Pregunta 4 (15 %)

La plataforma de eventos sociales ha ido incorporando nuevas funcionalidades alrededor de la gestión de eventos. Inicialmente, cerrar un evento consistía únicamente en cambiar el estado del *Event* a *CLOSED*. Sin embargo, con la evolución del sistema, el cierre de un evento implica ahora varias operaciones repartidas en distintos servicios.

Actualmente existen, entre otros, los siguientes componentes:

- *EventManager*: gestiona el estado del evento y sus inscripciones.
- *MatchingService*: calcula los *Match* a partir de los *Like* registrados.

Pero también tenemos estos otros componentes:

- *NotificationService*: envía mensajes a los participantes.
- *StatisticsService*: calcula métricas básicas del evento.

Cuando un organizador finaliza un evento, la aplicación debe ejecutar una secuencia completa de acciones:

1. Cambiar el estado del evento a *CLOSED*. Comprobar que el evento existe y está en estado válido para ser cerrado.
2. Generar los matches entre participantes y guardarlos.
3. Enviar notificaciones a los usuarios que han obtenido match.
4. Calcular estadísticas del evento.

Actualmente, esta lógica se ha implementado directamente en varios puntos del sistema. Cada cliente llama manualmente a los servicios anteriores y debe conocer el orden correcto de ejecución. Esto ha provocado multitud de problemas. Además, el equipo técnico prevé que en el futuro puedan añadirse nuevos pasos al cierre del evento, como publicar resultados en una app externa o enviar encuestas de satisfacción.

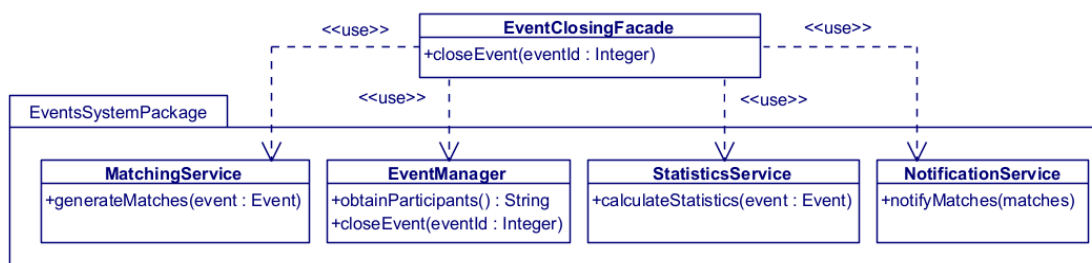
Pregunta 4.1 (5 %). Para mejorar el diseño, ¿qué patrón o combinación de patrones sería más adecuado para optimizar el comportamiento planteado en el enunciado? Justifica tu respuesta.

Pregunta 4.2 (10 %). Crea un diagrama de clases UML donde se refleje la solución propuesta. Se tienen que mostrar claramente las clases principales en la aplicación del patrón o patrones.

Solución

Pregunta 4.1. El patrón adecuado es **Facade**. Se aplica porque los clientes necesitan realizar una operación compleja —cerrar un evento— sin conocer todos los servicios internos ni el orden exacto de llamadas. El patrón **Facade** permite proporcionar una interfaz simple y unificada para una operación compleja que implica varios subsistemas. Los clientes no necesitan conocer *EventManager*, *MatchingService*, *NotificationService*, ni *StatisticsService*, sino que interactúan únicamente con *EventClosingFacade*. Esto reduce el acoplamiento, evita duplicación de lógica y facilita modificar el proceso de cierre sin cambiar a todos los clientes.

Pregunta 4.2.



Criterios de evaluación

La nota final de la PRA se calculará a partir de las notas obtenidas en cada una de las preguntas individuales, con el peso indicado en el enunciado. En concreto, este es el criterio:

- Pregunta 1: 25 %
- Pregunta 2: 30 %
- Pregunta 3: 30 %
- Pregunta 4: 15 %

Esta PRA corresponde a un **50% de la nota de Prácticas de la asignatura**.



RECUERDA: Para acogerte a la evaluación continua de la asignatura, deberás entregar todas las PECs y PRAs, y **superar cada una de ellas con al menos 3 puntos**.

Tal como se explica en el plan docente de la asignatura, esta actividad debe resolverse de forma individual. Por otro lado, no está permitido el uso de herramientas de inteligencia artificial. Recuerda que debes incluir el texto indicado en la sección “Compromiso de autorresponsabilidad” de tu PRA. En caso de detectar copias o uso de herramientas de IA, se penalizará la actividad con un 0 % como nota.

Formato de entrega

- Deberás entregar un **único documento** (formato pdf o docx) con tu propuesta de solución para las preguntas del enunciado.
- Recuerda que debes producir diagramas **conforme a la especificación UML** y **ser consistente** en el estilo de diagramas que entregues en todos los ejercicios.
- Todos los documentos se tienen que entregar en el aula virtual antes de las 23:59 horas del **9 de junio de 2026**.
- **No se aceptarán entregas fuera de plazo.**

Propiedad intelectual y plagio

La Normativa académica de la UOC dispone que el proceso de evaluación se cimenta en el trabajo personal del estudiante y presupone la autenticidad de la autoría y la originalidad de los ejercicios realizados.

La ausencia de originalidad en la autoría o el mal uso de las condiciones en las que se realiza la evaluación de la asignatura, es una infracción que puede tener consecuencias académicas graves.

Te recomendamos leer esta [guía sobre el plagio académico y como evitarlo](#). Recuerda que siempre que reutilices contenido de terceros, debes citar la fuente. Puedes leer también esta [guía para saber cómo citar correctamente](#).

El estudiante será calificado con un suspenso (0 %) si se detecta falta de originalidad en la autoría, sea porque haya utilizado material o dispositivos no autorizados, sea porque ha copiado textualmente de internet, o ha copiado apuntes, de otras PECs, de materiales, manuales o artículos (sin la cita correspondiente) o de otro estudiante, o por cualquier otra conducta irregular.